EE 533 Lab #8
Network Processor Integration
Young Cho - youngcho@isi.edu

NOTE: You must use GitHub to store all source codes and reports associated with the lab assignments. You must commit/push all modifications with detailed descriptions at least once daily while working on the lab assignment. Please submit the GitHub link and a record of your commits, along with detailed commit descriptions, to Coursistant before the due date.

1. Convertible FIFO

You must design and integrate a specialized FIFO to enable data flow between the NetFPGA reference pipeline designs and your ARM-compatible processor. The specifics of the FIFO, such as the widths of the data I/O and control interfaces, should depend on your processor and how you have chosen to interface with external components. However, it is expected that the FIFO be designed with Block RAM instantiated as dual-port SRAM, as in the Drop FIFO in the Mini-IDS laboratory.

One of the address and data ports can be used as the FIFO's input interface. The other port can be used as the output. As with the Drop FIFO, the addresses of the head and tail of the FIFO must be tracked using two accumulators, registers, and control logic. Although your FIFO does not have to perform the drop function, it is expected to identify the head and the tail of each packet, then (1) buffer all of the data and the control signal for one network packet, (2) when one packet is buffered in the FIFO, send a FIFO FULL signal to the circuit that is connected to the input so that next packet data is stalled, (3) send some sort of signal to the processor that the data is ready to be processed.

The processor should have read and write access to the head and tail address registers, and full access to the SRAM containing the data. One way to provide access to these components is to instantiate Multiplexors at the inputs and select between the Network FIFO and your processor's MEM stage in the pipeline. You can modify the design so your processor can interface with the FIFO. The following is the block diagram of the described FIFO.

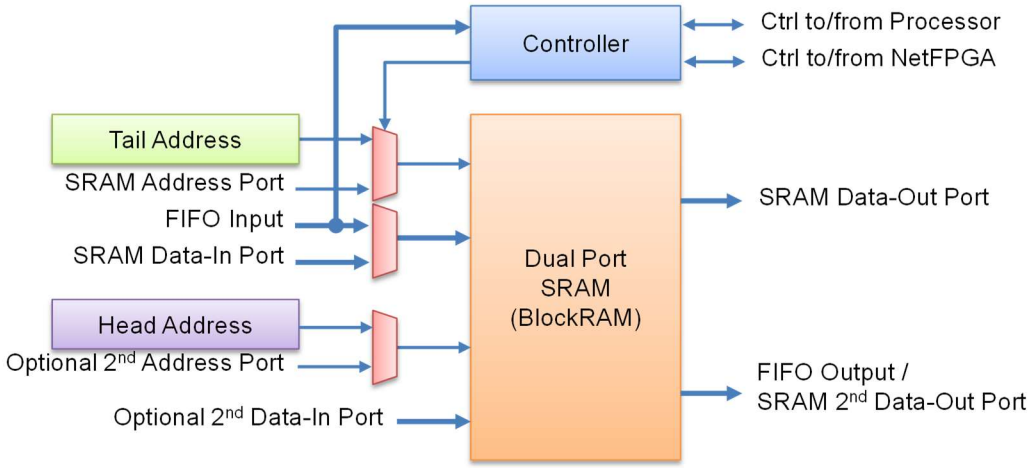

Figure 1: Convertible FIFO/Dual Port SRAM

Once the design is simulated, you must integrate and test the FIFO with the processor in NetFPGA. As part of the Lab, you must write and execute a few small codes to test your single-

core Network Processor. Your code should attempt to test reading and modifying the content of the network packets. You should also test the limits of your network processor, such as the maximum throughput of the code, etc.

One example test code might be to send network packets with integers at specific byte offsets of the payload. Then, your code can read from that payload offset, add a constant to the value, and write the result back to the same location. Once the packet is changed, you might revert the module to FIFO mode to send the modified packet to the destination IP address.

2. Integrating the GPU into the System

Once the FIFO is successfully integrated into either the reference IP router design or the reference network interface card design, you can augment the convertible FIFO with additional multiplexers and demultiplexers to give the GPU direct access to the memory content.

The following diagram is one example of design augmentation.

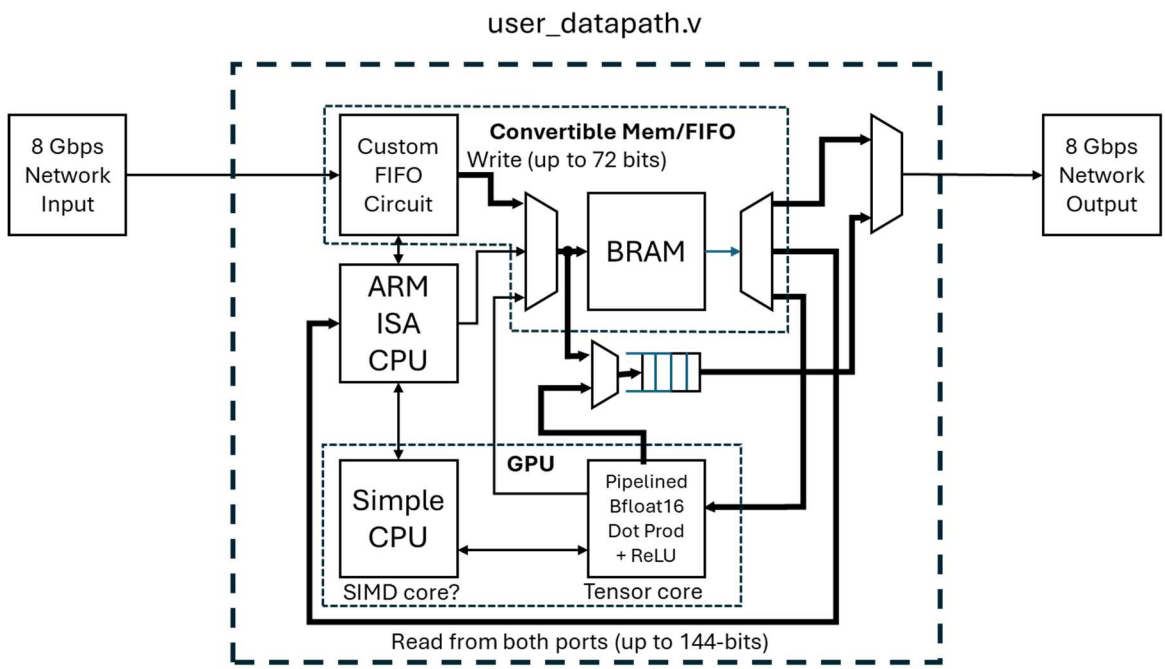

Figure 2: Network Interface with Network Processor and Graphics Processing Unit

In this example, the Block RAM used in the FIFO can function as a data memory for the ARM and GPU. Since a single Block RAM can be configured as a dual-port 256-by-72-bit-wide data memory, it can provide up to 128-by-144 bits of data per read clock cycle by reading 72-bit data from one port at address N and 72-bit data from the second port at address N+1, thereby doubling the memory data bandwidth. Once the GPU completes tensor operations, the result can either be written back to the Block RAM or forwarded to the FIFO output right after the packet's header fields. The goal should be to send the GPU result out as a network packet. You need to write some initial test code to execute simple tensor operations.

Please submit the source code, simulation output, test code, and a description of your test setup and results. You must also demonstrate test code running on your network processor on the demo YouTube video.